

Module 3

Bitcoin techniques

How to store and use Bitcoins

The important preconditions to use Bitcoins are:

- be able to spend your own money when you want to
- nobody else can spend your money
- convenience: the process must be relatively simple and don't cost too much

Possible approaches:

- **+ convenience**: the money could be accessible just pushing a button
- **- availability**: if the device crashes or you lose it, the money is lost with it
- **- security**: if someone manages to break into the device the security is broken. It's just as safe as carrying money in the pocket.

- **How to store cold info**

- (1) Info stored in device, device locked in a safe

- (2) “Brain wallet”

- encrypt info under passphrase that user remembers

- (3) Paper wallet

- print info on paper,

- lock up the paper

- (4) In “tamperproof” device

- device will sign things for you, but won't divulge keys

Paper wallet



Online wallets

Advantages:

- it is not necessary to install software to use a web based wallet, or a simple app on the phone
- it works across multiple devices. Even if a device is lost, the wallet will still be available

Disadvantages:

- if the website or app is malicious or gets compromised somehow, the Bitcoins can be lost. It is necessary to trust the website as more secure than oneself.



A Trezor wallet



Summary

Hot Storage:

- Connected to the internet
- Higher risk of hacking and theft
- - Used for frequent transactions

Cold Storage:

- - Not connected to the internet
- - Lower risk of hacking and theft
- - Used for long-term storage and security

Problem:

Want to use a new address (and key) for each coin sent to cold

But how can hot wallet learn new addresses if cold wallet is offline?

Better solution:

Hierarchical wallet

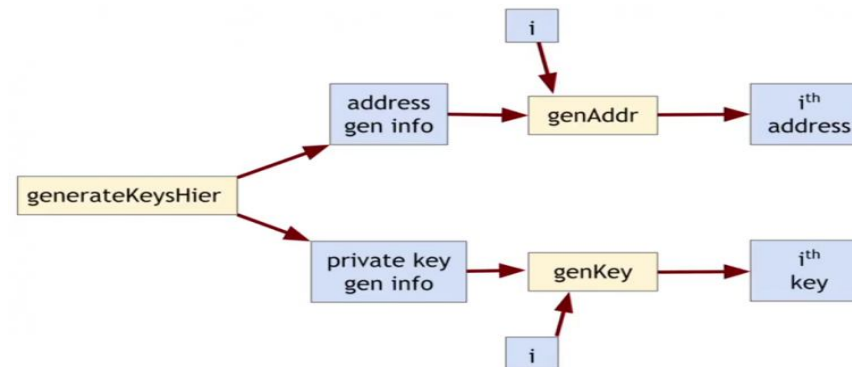
• Hierarchical key generation

The hierarchical key generates two things:

- **address generation info**
- **private key generation info**

These information can be used to generate multiple keys:

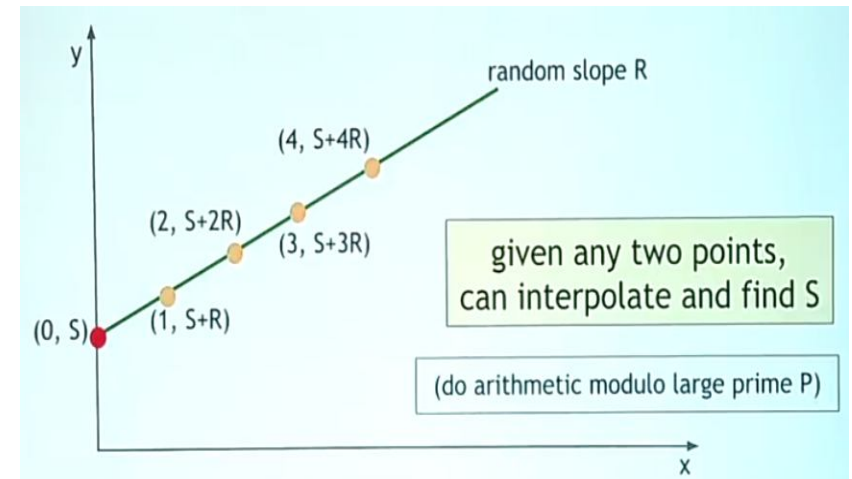
- given the address generation info, we can apply genAddr operation to the address generation info and integer i , obtaining the i th address. Since we can do this for every integer i , we can obtain a full sequence of addresses. This operation can be done on the public side.
- the same can be applied for private keys. So on the private side we can generate a sequence of private keys using genKey



• Split and share keys

How to increase N , with $K=2$

The idea to increase the number of shares is to use a line:



- take the 2D plane with X and Y axis
- choose a random value R
- draw a line with slope R and passing through the point $(0, S)$
- the shares will be the points on the line $(1, S+R)$, $(2, S+2R)$, $(3, S+3R)$, ...
- Clearly it is possible to choose as many shares as wanted, since there's an infinite number of points on the line
- Given any two points on a line, it is possible to retrieve its equations using the interpolation, so $K=2$.
- Given just one point, it's impossible to retrieve any information about the line
- If we do this operation using the arithmetic modulo a large prime P , all we have said is still applicable.

How to increase K

If we want to increase K, we can use functions that require more than two points to be defined.

For example, if we want $K=3$, we will use a quadratic function. In fact, three points are necessary to define uniquely a quadratic function. In this case, there will be two random parameters R_1 and R_2 , since the equation would be:

$$Y = R_2 X^2 + R_1 X + S$$

And we can go on like this increasing the polynomial degree.

For example, if we want $K=4$ we can use a cubic function with three random parameters and so on. Every curve use has infinite points, so we can also increase N at will.

Equation	Random parameters	Points needed to recover S
$(S + RX) \bmod P$	R	2
$(S + R_1X + R_2X^2) \bmod P$	R_1, R_2	3
$(S + R_1X + R_2X^2 + R_3X^3) \bmod P$	R_1, R_2, R_3	4

Example: Shamir's secret-sharing scheme

Suppose we want to use a (k, n) threshold scheme to share our secret S size

Assumed to be an element in a finite field F of size P where $0 < k \leq n < P$; $S < P$ and P is a prime number.

Secret Sharing

- A (k, n) threshold polynomial can be written by
$$s(x) \equiv M + s_1x + s_2x^2 + \dots + s_{k-1}x^{k-1} \pmod{p}$$
- Deliver $(x_i, s(x_i))$ to the i -th participant

p = a (large) prime number

M : secret value

$s_1, \dots, s_{k-1}$: randomly chosen from $[0, p-1]$

Secret Sharing...

- To reveal the secret value M , we must collect (at least k) $\geq k$ shadows.
- Without loss of generality, we use $(x_1, s(x_1)), \dots, (x_k, s(x_k))$ as k shadows.
- We can reveal the secret value M by using Lagrange interpolation.

$$s(x) \equiv \sum_{j=1}^k \left[s(x_j) \prod_{i=1, i \neq j}^k \frac{x - x_i}{x_j - x_i} \right] \bmod p$$

- where $M = s(0)$

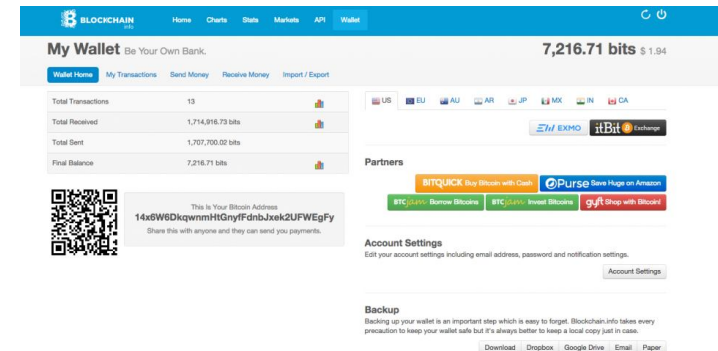
Secret sharing Pros and Cons:

it is possible to store the shares separately. So an adversary needs to be able to retrieve K shares in order to get back the secret key. So, he needs to break the security of different places K times. In addition, we can afford to lose some shares, since only K of them chosen randomly are needed to retrieve the key.

if we want to sign a transaction, we need to reconstruct the key bringing the shares together. This is a point of vulnerability. If we get attacked at this moment the adversary can recover the full key.

• Online wallets and exchanges

Online wallets



Advantages:

- it is not necessary to install software to use a web based wallet, or a simple app on the phone
- it works across multiple devices. Even if a device is lost, the wallet will still be available

Disadvantages:

- if the website or app is malicious or gets compromised somehow, the Bitcoins can be lost. It is necessary to trust the website as more secure than oneself.

- **Cryptocurrency exchanges**

Bitcoin exchanges are businesses that, at least from a user interface point of view, are similar to banks:

- accept deposits of Bitcoins and fiat currency (es. ₹, \$, €, ...)
- enable to send and receive Bitcoins
- buy and sell Bitcoins for fiat currency. Typically they try to match Bitcoin buyers and sellers. So they find some customer who wants to buy bitcoins with Rupees/dollars/euros and some other customer who wants to sell bitcoins for dollars/euros and try to match them up.

Exchange buy and sell process

- **Exchange risks**

It is really easy to trade back and forth from fiat currency and Bitcoins using an Exchange.

The risky aspect is that it behaves such as a bank, so:

- risk of **bank run**.
- **Ponzi scheme**
- risk of a **cyber attack**



- **Proof of reserve**

There are two components to the Proof of Reserve:

- prove **how much reserve** it's holding.
- prove **how many demand deposits** the group holds.

To get the amount of **fractional reserve** divide those two numbers by themselves

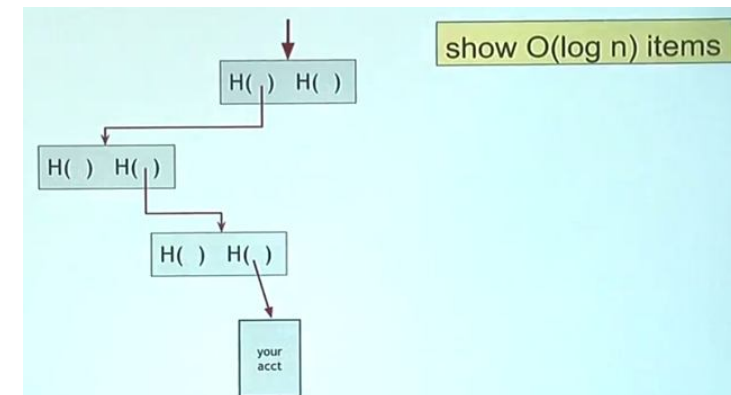
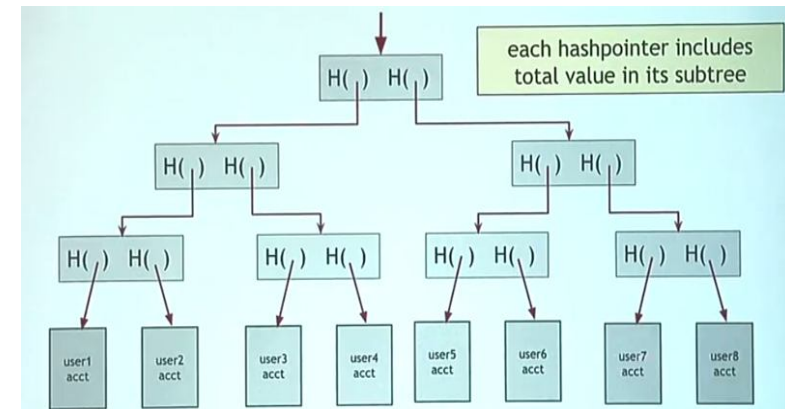
Prove the reserve amount

The company proves it owns a certain amount of BTC by doing the following:

- It creates a transaction that sends the claimed amount of BTC to itself (e.g., 100,000 BTC).
- This self-payment transaction uses the same private key that was used to sign a random string provided by an unbiased third party..
- This proves that the person with the private key is participating in the proof of reserve .
- It shows that the company or someone cooperating with the company controls at least the claimed amount of BTC.
- The company might prove ownership of 100,000 BTC even if it actually has more (e.g., 150,000 BTC).

Prove the number of deposits

- **Create a Merkle Tree:** Build a binary tree using hash pointers, which tell where the information is and its hash.
- **Label Pointers with Values:** Add a label to each hash pointer with the total Bitcoin value of everything under that pointer in the tree.
- **Place Customer Accounts:** Put all customer accounts at the bottom of the tree.
- **Combine Accounts:** Combine the accounts into a Merkle tree, so each node's hash pointer is labeled with the sum of the two pointers below it.
- **Root Represents Total Deposits:** The root of the tree will represent the total deposit amount.
- **Sign the Root:** The exchange signs the root of the tree to validate it.
- **Customer Verification:** Each customer can request to see their inclusion in the tree.
- **Show Path to Account:** The exchange shows the path to the customer's account.
- **Check Consistency:** Customers check that the hash pointers are consistent and that their deposit adds up to the total.
- **Verify All Branches:** If all customers do this, every branch of the tree is explored and verified.



Conclusion:

If the two pieces of the proof gets correctly verified, then the exchange has proven that:

- They own an amount of at least X Bitcoin of reserve
- The total deposits of the customers correspond to Y

So the reserve fraction would be X/Y , and it can be compared to what's declared by the company. These information can be independently verified by anybody, so it's a proof that can increase the trust in the exchange.

- **Multi signature Wallets**

Example (2-of-3 multisig):

plaintext

Multi signature Wallets

 Copy code

```
2 <PublicKey1> <PublicKey2> <PublicKey3> 3 OP_CHECKMULTISIG
```

- **Payment services:**

The merchant's requirements are:

- At the end of the day he wants to receive **fiat currency**
- have a way of receiving payments in Bitcoins which is **easy to deploy**. So he doesn't have to worry about technology changing on its website or build new infrastructure.
- have **low risk**. The merchant wants to avoid problems coming from a technology change, for example website problems or collapse. In addition he wants to avoid risks that someone breaks into his hot wallet. And he doesn't want to deal with the exchange rate risk, since the Bitcoin value in dollars may fluctuate from time to time. So if the value of a good is 100\$, he wants to receive an amount very close to it.
- The payments services allow both the customers and the merchants to be happy about the payments in Bitcoins.

Payment process

Here's what happens when a customer chooses to pay with Bitcoin using the new button:

- the merchant delivers a page that contains the "**Pay with Bitcoin**" button. The button will contain a **transaction ID**, an identifier meaningful to the merchant in its own accounting system, along with an **amount** to be paid.
- The information that the button has been clicked will be sent to the payment service, along with the transaction id, the amount and merchant's identity.
- The payments service knows that the customer wants to pay with Bitcoins, so will start an interaction with the user to give information about how to pay.
- When the user confirms the payment, there will be a redirect that goes back to user's browser. This will also send a message to the merchant, saying that for the payment service everything is ok until then.
- Later, the payment service will directly send a confirmation to the merchant, when it's fully confirmed by the blockchain.
- The merchant can now send the product to the customer.

- **Transaction fees**

Transaction fee= output value-input value

Current consensus fee

Let's see how is the current consensus fee:

- **no fee** if:
 - The transaction is less than 1000bytes in total size
 - All the outputs of the transaction are at least 0.01 BTC
 - The priority of the transaction is large enough, where **Priority** = $(\text{input_age}_1 * \text{input_value}_1 + \text{input_age}_2 * \text{input_value}_2 + \dots + \text{input_age}_N * \text{input_value}_N) / \text{transaction_size}$

- **input_value_in_base_units** is the number of satoshis that the input is worth. One Bitcoin is worth 100,000,000 satoshis.
- **input_age** is how many blocks the input has been present for. An unconfirmed transaction has an age of 0, and one that has 100 confirmations has an age of 100.

- a fee of 0.0001 Bitcoins every 1000 Bytes of the transaction. The transaction size is approximately 148 bytes per each input plus about 34 bytes per each output plus about 10 bytes for other information

So, even if these consensus rules are not followed, the transaction will probably find its way into the blockchain sooner or later. This is ok if you don't need it to be approved quickly.

Most online wallets and payment services include automatically a fee following those rules to forward reliable transactions. So you'll see a little bit of money racked off for transaction fees, when you make payments.